



Chapter 2: Literature Review

Created by	 Aroosh Ahmad
Created time	May 18, 2026 7:48 AM
Last edited by	 Aroosh Ahmad
Last updated time	May 18, 2026 8:06 AM

2.1 Overview

This chapter reviews the literature relevant to instruction fine-tuning of large language models, parameter-efficient fine-tuning, curriculum learning, semantic similarity-based grouping, mini-batch construction, and evaluation metrics for generated text. The purpose of this review is to position the present study within existing research and identify the gap addressed by this thesis: the role of mini-batch construction and batch scheduling in instruction fine-tuning.

Existing work in instruction fine-tuning has mainly focused on improving model alignment, scaling instruction datasets, selecting or curating training data, and designing curriculum strategies at the dataset level. However, the role of mini-batch construction itself remains less explored. In standard supervised fine-tuning, mini-batches are usually created randomly or through simple heuristics, implicitly assuming that batch composition does not substantially affect learning dynamics.

This thesis challenges that assumption by studying whether semantic mini-batch construction and curriculum-based batch scheduling influence optimization, generalization, and generation quality.

2.2 Large Language Models and the Transformer Architecture

Modern large language models are primarily built on the Transformer architecture. Vaswani et al. introduced the Transformer as an attention-based architecture that removed the need for recurrence and convolution in sequence modeling [1]. Its self-attention mechanism enables the model to capture relationships between tokens efficiently and in parallel, making it highly suitable for large-scale language modeling and sequence-to-sequence learning.

The success of the Transformer architecture led to a major shift in natural language processing. Instead of training separate models for individual tasks, researchers increasingly adopted pre-training and fine-tuning pipelines. In this paradigm, a model is first trained on large-scale general text data and later adapted to downstream tasks.

Raffel et al. introduced T5, which framed diverse NLP tasks within a unified text-to-text format [2]. In T5, both input and output are represented as text. This formulation is particularly relevant to instruction fine-tuning because instructions and responses can naturally be modeled as text sequences.

The present study uses Flan-T5-small as the base model. Flan-T5 belongs to the family of instruction-fine-tuned T5 models. Chung et al. showed that instruction fine-tuning across diverse tasks improves model performance and generalization across several settings, and they also released Flan-T5 checkpoints as part of this work [3]. Flan-T5-small is therefore suitable for controlled experiments because it is publicly available, computationally manageable, and designed for instruction-following behavior.

2.3 Instruction Fine-Tuning

Instruction fine-tuning adapts a pre-trained language model using datasets composed of instructions and corresponding responses. Instead of learning only from raw language modeling objectives, the model is trained to respond to task descriptions, user queries, and natural language commands. This makes instruction fine-tuning central to building language models that are more useful in practical settings.

Ouyang et al. introduced InstructGPT and showed that language models can be better aligned with user intent through supervised fine-tuning and reinforcement learning from human feedback [4]. Their work demonstrated that increasing

model size alone does not guarantee better instruction-following behavior. Instead, training procedures and alignment data play an important role in making models more helpful and reliable.

The FLAN line of work further demonstrated that fine-tuning models on a wide range of instruction-formatted tasks improves zero-shot and few-shot generalization. Chung et al. studied instruction fine-tuning across model size, task scale, and chain-of-thought data, showing that instruction fine-tuning is a general method for improving the usability of pre-trained language models [3].

Datasets used for instruction fine-tuning are typically heterogeneous. They may include question answering, summarization, classification, brainstorming, information extraction, open-ended generation, and reasoning-style tasks. This heterogeneity is important for the present research. If instruction-tuning datasets contain diverse task types, then random mini-batches may provide useful task mixing. By contrast, semantically grouped batches may reduce within-batch diversity. This makes the effect of semantic batching non-trivial and motivates empirical evaluation rather than assuming that semantic grouping must improve performance.

2.4 Parameter-Efficient Fine-Tuning and LoRA

Fine-tuning all parameters of a large language model is expensive in terms of computation, memory, and storage. Parameter-efficient fine-tuning methods address this issue by updating only a small subset of parameters or introducing additional trainable components while keeping most of the base model frozen.

Low-Rank Adaptation, commonly known as LoRA, is one of the most widely used parameter-efficient fine-tuning methods. Hu et al. proposed LoRA as a technique that freezes the pre-trained model weights and injects trainable low-rank matrices into model layers [5]. This significantly reduces the number of trainable parameters and memory requirements while maintaining competitive downstream performance.

LoRA is especially useful for experimental research because it allows multiple fine-tuning runs to be conducted under practical hardware constraints. Since this thesis compares several batching strategies across dataset sizes and random seeds, full fine-tuning would be computationally expensive. LoRA therefore

provides a practical fine-tuning mechanism while preserving the ability to compare training strategies fairly.

In this study, LoRA is used with Flan-T5-small across random batching, semantic grouping, mixed batching, curriculum scheduling, and length-based curriculum experiments. The goal is not to propose a new parameter-efficient fine-tuning method, but to use LoRA as a controlled and efficient fine-tuning approach for studying the effect of batch scheduling.

2.5 Mini-Batch Training and Random Batching

Mini-batch training is a standard component of neural network optimization. Instead of computing gradients over the entire dataset at once, training is performed on smaller batches of examples. These mini-batches provide stochastic gradient estimates that make training computationally feasible and often improve generalization.

In most supervised fine-tuning pipelines, mini-batches are formed randomly. Random batching has several practical advantages. It avoids systematic ordering bias, exposes the model to diverse examples within each training phase, and produces gradient estimates that are less tied to a narrow region of the data distribution. In heterogeneous instruction-tuning datasets, random batching may also act as a form of implicit regularization by mixing different task types.

This point is directly relevant to the present study. The experimental results later reported in this thesis show that random batching remains a strong and reliable baseline across multiple dataset scales. In the main 5K experiments, random batching performs competitively against grouped and curriculum-based variants. This observation is important because it challenges the assumption that more structured batching must automatically improve learning.

2.6 Semantic Similarity and Embedding-Based Grouping

Semantic batching relies on the idea that similar instructions should be trained together. To implement such grouping, instruction texts must first be converted

into vector representations. Sentence embedding models provide a practical way to represent sentence-level meaning in dense vector space.

Reimers and Gurevych introduced Sentence-BERT, a modification of BERT using Siamese and triplet network structures to produce semantically meaningful sentence embeddings [6]. These embeddings can be compared using cosine similarity and are more efficient for semantic search and clustering than using standard BERT directly for pairwise sentence comparisons.

Once sentence embeddings are generated, nearest-neighbor search can be used to identify semantically similar examples. Johnson et al. introduced FAISS, a library for efficient similarity search and clustering over dense vectors [7]. FAISS makes it practical to retrieve nearest neighbors from large embedding collections, which is useful for constructing semantically coherent mini-batches.

In the present study, semantic batching is implemented using sentence embeddings and FAISS-based nearest-neighbor retrieval. Instructions that are close in embedding space are grouped into semantically coherent mini-batches. The motivation behind this strategy is that semantically related examples may produce more aligned gradients, potentially improving optimization or generalization.

However, the empirical results of this thesis show that this intuition does not consistently hold. Semantic batching influences batch properties by increasing coherence and reducing diversity, but these structural differences do not translate into meaningful improvements in evaluation loss or generation metrics. Therefore, this study treats semantic grouping as a hypothesis to be tested rather than as an assumed improvement.

2.7 Curriculum Learning

Curriculum learning is based on the idea that the order of training examples can influence learning. Bengio et al. introduced curriculum learning as a training strategy where models are exposed to examples in a meaningful order, often moving from easier examples to harder ones [8]. The central idea is that a carefully designed order of examples may guide optimization and improve learning.

Curriculum learning is relevant to this thesis because it shifts attention from static batch composition to training order. Instead of asking only whether semantically similar samples should be grouped together, this study asks when structure should be introduced during training and whether ordering affects learning dynamics.

The experiments in this thesis consider three curriculum-related strategies. First, a diversity-to-focus strategy trains the model using random batching in the first phase and grouped batching in the second phase. Second, an anti-curriculum strategy reverses this order by training with grouped batches first and random batches later. Third, a length-based curriculum uses instruction length as a proxy for difficulty and compares easy-to-hard and hard-to-easy schedules.

This design directly connects curriculum learning theory with instruction fine-tuning practice. Rather than applying curriculum only at the dataset level, the present study investigates whether curriculum-style scheduling at the mini-batch level can influence fine-tuning behavior.

2.8 Data Ordering, Training Dynamics, and Path Dependence

Neural network training is path-dependent. The sequence in which examples are presented can influence the optimization trajectory, especially during early training stages. Early gradient updates shape the region of parameter space explored by the model, and later updates refine or modify that trajectory.

This path dependence motivates the two-phase experiments in this thesis. The random-to-grouped schedule can be interpreted as an exploration–refinement strategy. In the first phase, random batching exposes the model to diverse examples and encourages broad task learning. In the second phase, grouped batching provides more focused updates. By contrast, grouped-to-random scheduling may introduce structure early but later dilute that structure through random mixing.

The controlled experiments reported later in the thesis show a consistent directional pattern: random-to-grouped performs better than grouped-to-random in the early multi-seed setting. The numerical differences are small, but the direction supports the claim that training order can influence learning dynamics.

This motivates a broader interpretation that effective fine-tuning may require a balance between diversity and structure.

2.9 Difficulty-Based Curriculum and Length as a Proxy

Traditional curriculum learning often assumes that training should move from easier to harder examples [8]. In language tasks, however, difficulty can be approximated in different ways, including instruction length, response length, embedding density, model loss, or task category.

This thesis uses instruction length as a simple difficulty proxy. The assumption is that longer instructions may contain more constraints, more detailed context, or more complex task requirements. Based on this proxy, two curriculum schedules are considered: easy-to-hard and hard-to-easy.

Interestingly, the thesis results show that hard-to-easy length-based curriculum performs better than easy-to-hard curriculum under the tested setup. This does not prove that hard-to-easy scheduling is universally superior, but it suggests that reverse curriculum may be beneficial in this particular instruction fine-tuning setting. One possible explanation is that early exposure to complex examples helps the model learn broader representations, while later easier examples stabilize and refine learning.

This finding redirects the study from semantic similarity alone toward difficulty-aware scheduling. It also supports the broader argument that curriculum design may be more useful than static semantic grouping for instruction fine-tuning.

2.10 Evaluation Metrics for Instruction Fine-Tuning

Evaluating instruction fine-tuning requires both optimization-based and generation-based metrics. Optimization metrics such as training loss and evaluation loss measure how well the model fits the training and validation data. However, lower loss does not always guarantee better generation quality. Therefore, text generation metrics are also needed.

ROUGE is a widely used automatic evaluation metric for text generation and summarization. Lin introduced ROUGE as a package for automatic evaluation of summaries using overlap-based measures such as ROUGE-N and ROUGE-L [9].

ROUGE measures lexical overlap between generated and reference text, which makes it useful but limited when semantically similar outputs use different wording.

BERTScore addresses some limitations of surface-level overlap metrics by using contextual embeddings from BERT to compare generated and reference text at the token level. Zhang et al. proposed BERTScore as a semantic evaluation metric for text generation and showed that it correlates better with human judgments than several traditional metrics [10].

This thesis uses ROUGE-1, ROUGE-2, ROUGE-L, and BERTScore F1 to evaluate generated responses. The experimental results show that generation metric differences across batching strategies are very small. This supports the conclusion that semantic batching does not meaningfully change output behavior under the tested conditions.

2.11 Research Gap Summary

The reviewed literature establishes several important points. First, Transformer-based language models provide the architectural foundation for modern LLMs [1]. Second, instruction fine-tuning improves the ability of language models to follow user intent and generalize across tasks [3], [4]. Third, parameter-efficient methods such as LoRA make repeated fine-tuning experiments feasible under limited computational resources [5]. Fourth, semantic embeddings and vector search tools such as Sentence-BERT and FAISS make semantic grouping technically feasible [6], [7]. Finally, curriculum learning provides a theoretical basis for studying training order [8].

However, the specific role of mini-batch construction in instruction fine-tuning remains underexplored. Existing work has paid more attention to model scaling, instruction dataset construction, data curation, and alignment. Less attention has been given to whether the composition of individual mini-batches or the order of batching strategies across training phases affects fine-tuning behavior.

This thesis addresses that gap by systematically comparing random batching, semantic grouping, mixed batching, curriculum-based ordering, and length-based curriculum schedules. The study does not assume that semantic grouping must improve performance. Instead, it empirically tests the hypothesis and finds that

semantic batching alone does not provide a consistent advantage over random batching. The study then redirects attention toward curriculum-based scheduling, especially difficulty-based ordering, which shows a small but more promising signal.

2.12 Summary

This chapter reviewed the literature on large language models, instruction fine-tuning, LoRA, mini-batch training, semantic similarity, curriculum learning, and evaluation metrics. Prior work shows that instruction fine-tuning is central to making language models follow user intent, while LoRA enables efficient adaptation. Semantic embeddings and vector search provide the tools required for semantic grouping, and curriculum learning provides a theoretical foundation for studying training order.

The key gap identified is that mini-batch construction is usually treated as an implementation detail rather than a research variable in instruction fine-tuning. This thesis contributes by examining whether semantic and curriculum-based batch scheduling strategies affect optimization and generation quality. The literature supports the plausibility of the hypothesis, while the experimental results of this thesis provide a careful empirical test of its limits.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 2017.
- [2] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, 2020.
- [3] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, Y. Li, X. Wang, M. Dehghani, S. Brahma, et al., "Scaling Instruction-Finetuned Language Models," *Journal of Machine Learning Research*, 2024.
- [4] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al., "Training Language Models to Follow

Instructions with Human Feedback," *Advances in Neural Information Processing Systems*, 2022.

[5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations*, 2022.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.

[7] J. Johnson, M. Douze, and H. Jégou, "Billion-scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, 2019.

[8] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum Learning," *Proceedings of the 26th Annual International Conference on Machine Learning*, 2009.

[9] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," *Text Summarization Branches Out*, Association for Computational Linguistics, 2004.

[10] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," *International Conference on Learning Representations*, 2020.